Express Middleware

- Express Middleware
  - Qué es middleware?
  - Declaración de middleware
  - Qué hace el middleware?
  - Módulos npm
  - Encadenamiento de middleware
  - Partes de una función Middleware
  - Tipos de middleware
    - Middleware incorporado
    - Middleware de aplicación
    - Middleware de direccionador
    - Middleware de manejo de errores
    - Middleware de terceros
  - Middleware Ejemplo 1
  - Middleware Ejemplo 2

Qué es middleware?

Se refiere al código, mas específicamente a las funciones que se ejecutan durante el ciclo de solicitud-respuesta en un servidor de Express.

Las funciones middleware se ejecutan:

- Antes de que una petición entre a un manejador de rutas.
- Antes de enviar la respuesta al cliente.

Las funciones middleware tienen acceso a:

- El objeto `request`
- El objeto `response`
- La siguiente función middleware (denotada por `next`)

Las funciones middleware pueden realizar lo siguiente:

- Pueden ejecutar código.
- Pueden cambiar los objetos `request` y `response`
- Pueden finalizar el ciclo `request-response` (de no finalizarlo la solicitud queda colgada)
- Pueden invocar a la siguiente función middleware (invocando `next()`)

Una función middleware debe hacer una de dos cosas, para que nuestra app Express no se quede ciclada, siempre debe:

- Cerrar el ciclo de solicitud-respuesta. `res.send()`
- Pasar el control a la siguiente función middleware. `next()`

Express Middleware | ⬅️ | ➡️

Declaración de middleware

Usando el método **use**, pasándole una función, esta función se ejecutará para todos los métodos HTTP y todas las rutas.

```
const express = require('express');
const app = express();

app.use(function(req,res,next) {
    // Código de la función middleware
    next();
});
//Implementar Manejo de rutas
```

Filtrando según el tipo de petición y una ruta determinada:

```
const express = require('express');
const app = express();

app.use('/ruta',function(req,res,next) {
    //Código de la función middleware
    next();
});

app.get('/ruta',function(req,res,next) {
    //Código de la función middleware
    next();
});
```

Express Middleware | ⬅️ | ➡️

Qué hace el middleware?

Cuando llega una petición al servidor Express, esta puede pasar a traves de diversas funciones middleware hasta llegar al núcleo de nuestra aplicación, estas funciones pueden realizar diversas acciones como:

- Verificación de acceso.
- Validación de datos.
- Manejo de errores.
- O simplemente ejecutar código.

Módulos npm

En el registro npm de Javascript existes módulos que podemos agregar a nuestro servidor y que nos dan funciones Middleware para hacer muchas y muy diversas funciones

Función	Módulos npm
Seguridad	helmet, express-rate-limit, xs-clean
Parseo	express-urlencoded, cookie-parser, multer
Validación	express-validator, Joi, Zod
Autenticación	express-session, passport
Rendimiento	compression

Express Middleware | ⬅️ | ➡️

Encadenamiento de middleware

En el método **`app.use()`**, podemos encadenar varias funciones middleware y la solicitud pasará a traves de ellas.

```
app.use( ()=>{ }, ()=>{}, ()=>{ })
```

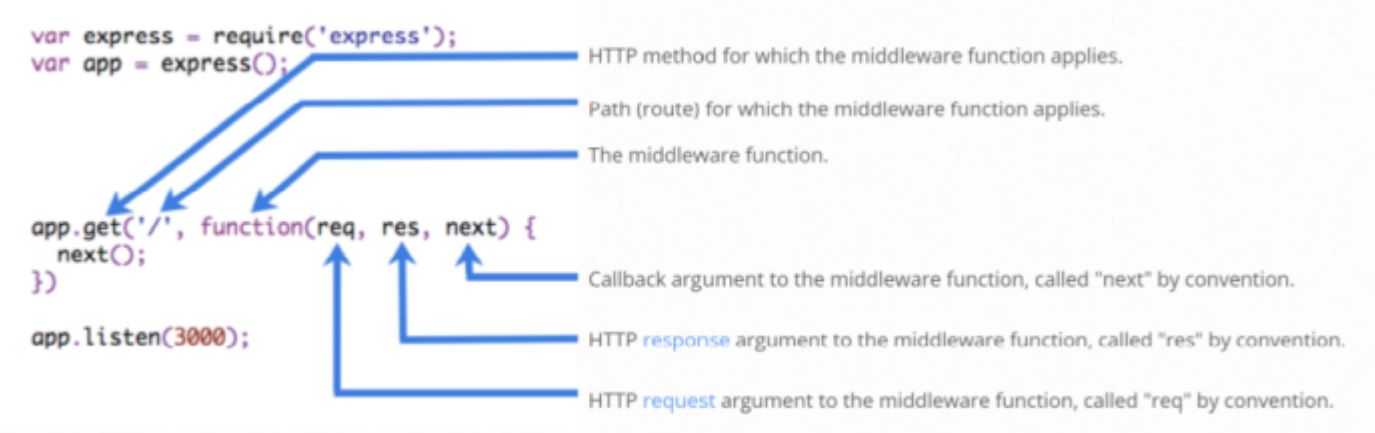
Recordemos que las funciones middleware reciben el objeto **`request`**, el objeto **`response`**
y una referencia a la siguiente función middleware con **`next`**.

```
const express = require('express');
const app = express();

app.use((req,res,next) => {
      console.log('Primer función middleware')
      next();
},(req,res,next) => {
     console.log('Segunda función middleware')
     next();
})
//Implementar Manejo de rutas
```

Express Middleware | ⬅️ | ➡️

Partes de una función Middleware



Referencia de la página oficial de Express

Tipos de middleware

- **Middleware incorporado** Funciones middleware que ya vienen por default con Express
- **Middleware de aplicación** Funciones middleware que se ejecutan a nivel global antes de las rutas
- **Middleware de direccionador** Funciones middleware que están ligadas a un Router
- **Middleware de manejo de errores** Función middleware que se encargará de manejar los errores
- **Middleware de terceros** Funciones middleware contenidas en módulos de Js que podemos instalar

Express Middleware | ⬅️ | ➡️

Middleware incorporado

Son las funciones Middleware incorporadas por default en el servidor Express, opr ejemplo

- **`express.static()`** entrega e contenido de una carpeta en el servidor, por convención es **`/public`**
- **`express.json()`** Si el contenido del body de la petición es Json, lo agregará a la propiedad **`body`** del objeto **`request`**

```
app.use("/public", express.static(path.join(__dirname,'public'))  );
```

```
app.use(express.json());
```

Express Middleware | ⬅️ | ➡️

Middleware de aplicación

Son las funciones que se ejecutan a nivel global antes de llegar a las rutas y que se ejecutan para todas las solicitudes:

```
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.txt());

/*
Implementación del manejo de rutas
*/

app.listen(3000,()=>{
  console.log('Servidor Express escuchando en puerto 3000')
});
```

Middleware de direccionador

Funciones ligadas a una instancia de **`express.router()`**

```
const router = require('express').Router();

router.use(function (req, res, next) {
  /*
  Función middleware del router
  */
 });


/*
Implementación del manejo del recurso del router
*/

module.exports.router=router;
```

Express Middleware | ⬅️ | ➡️

Middleware de manejo de errores

Justo después del manejo de las rutas suele incluirse una función middleware que sera la encargada
de regresar al cliente el error Http 404, en caso de que no se haya ejecutado ningún manejador de ruta.
Y después una función middleware para detectar errores y enviar mensajes personalizados de error al cliente.

```
/*
Manejo de las rutas
*/

app.use(function(req, res, next) {                // Función al final del manejo de rutas para error 404
  res.status(404).send('Recurso no encontrado');
});

app.use(function(err, req, res, next) {           // Función personalizada de manejo de errores
  res.status(500).send('Algo no ha ido bien');
});

app.listen(8000,function() {                      // Enciende el servidor y lo pone a escuchar en el puerto 8000
    console.log("Servidor escuchando");
});
```

Express Middleware | ⬅️ | ➡️

Middleware de terceros

Son módulos de otros desarrolladores o que instalamos del registro npm
que cumplen con alguna tarea específica. **`cors(); morgan();`**

- **Módulo cors:** Habilita al servidor para que responda a peticiones de origen cruzado
- **Módulo morgan**: Lleva un registro o log de las peticiones hechas al servidor

```
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());

/*
  Implementación del manejo de rutas
*/
```

Express Middleware | ⬅️ | ➡️

Middleware Ejemplo 1

Implementación de una función Middleware para verificar que en una petición **POST** al servidor se esté enviando contenido Json.

```
const express = require('express');
const app = express();

const verificarJson = (req, res, next) => {
    if (req.headers['content-type'] !== 'application/json') {
        res.status(400).send('Servidor solo acepta Json')
    } else {
      next();
    }
  }


app.post('/recurso', verificarJson, (req, res, next) => {
  res.send('Contenido Json Recibido');
})

app.listen(3000,()=>{ console.log('Servidor Express escuchando en puerto 3000')});
```

Express Middleware | ⬅️ | ➡️

Middleware Ejemplo 2

Función Middleware para que el servidor acepte peticiones solo en un rango de Fechas

```
const express = require('express');
const app = express();

 app.use( (req, res, next) => {
      const fecha = new Date();

      if (fecha >= new Date('2022-01-01') &&
          fecha <= new Date('2022-01-10')) {
            next();                                   // Pasar la ejecución a la siguiente función Middleware
      } else {
            res.status(403);                          // Petición fuera de fechas regresar Err al cliente
            res.send('Petición fuera de horario');
      }
  };
 )

app.post('/', (req, res, next) => {
    res.send('Servidor contestando a petición POST');
})

app.listen(3000,()=>{ console.log('Servidor Express escuchando en puerto 3000')});
```

Express Middleware | ⬅️